

Tensors:

Cont'dn:

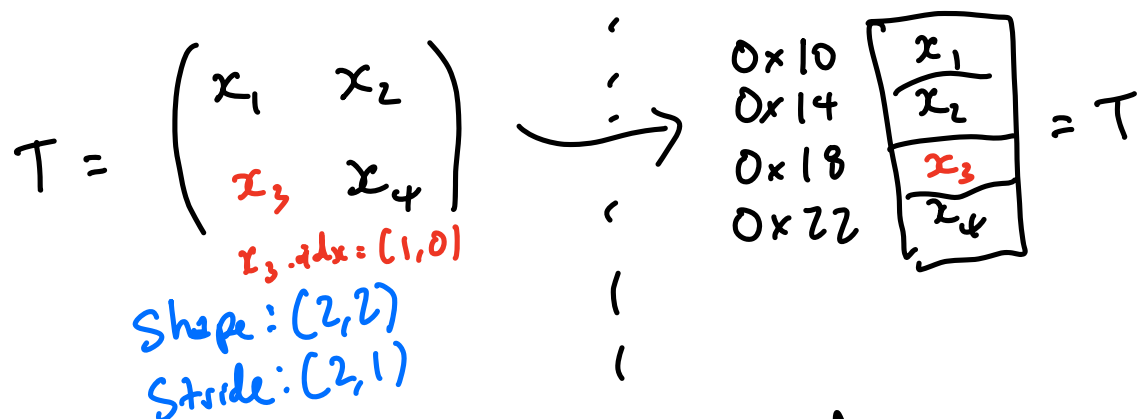
data: $\begin{bmatrix} \begin{bmatrix} x_{11} \\ \vdots \end{bmatrix} \\ \vdots \end{bmatrix}$

metadata: sizes, dtype, device, layout, strides?

Strides:

- Despite the 2d intuition of tensors, the easiest way to store them is as contiguous values in memory (so as a 1d structure):

Intuitive view of tensor: What is actually stored



Thus we need a system for mapping the 1d index to 2d.

We use Stride for this.

For this we define the stride for each dimension,

and use the lookup formula:

$$idx = stride.x * x + stride.y * y$$

$$idx = 2 * 1 + 0 * 1 = 2$$

This striding feature also enables the generation of views of tensors. For example, we can support this:

$$\text{tensor}[1,:] \rightarrow (x_3, x_4), \text{tensor}[:,1] \rightarrow (x_1, x_3)$$

To do this we return a new tensor object that holds the same underlying data vector but updates the internal "offset" and "stride" values:

$$T_1: \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

$$\text{data} = [1, 2, 3, 4] \quad (\text{stored as a c++ vec})$$

$$\text{stride} = [2, 1]$$

$$\text{offset} = 0$$

$$T_2 = T_1[1,:]: (3, 4)$$

$$\text{data} = [1, 2, \underline{3}, \underline{4}]$$

$$\text{stride} = [1]$$

$$\text{offset} = 2$$

$$T_3 = T_1[:,1] = (1, 3)$$

$$\text{data} = [\underline{1}, 2, \underline{3}, 4]$$

$$\text{stride} = [2]$$

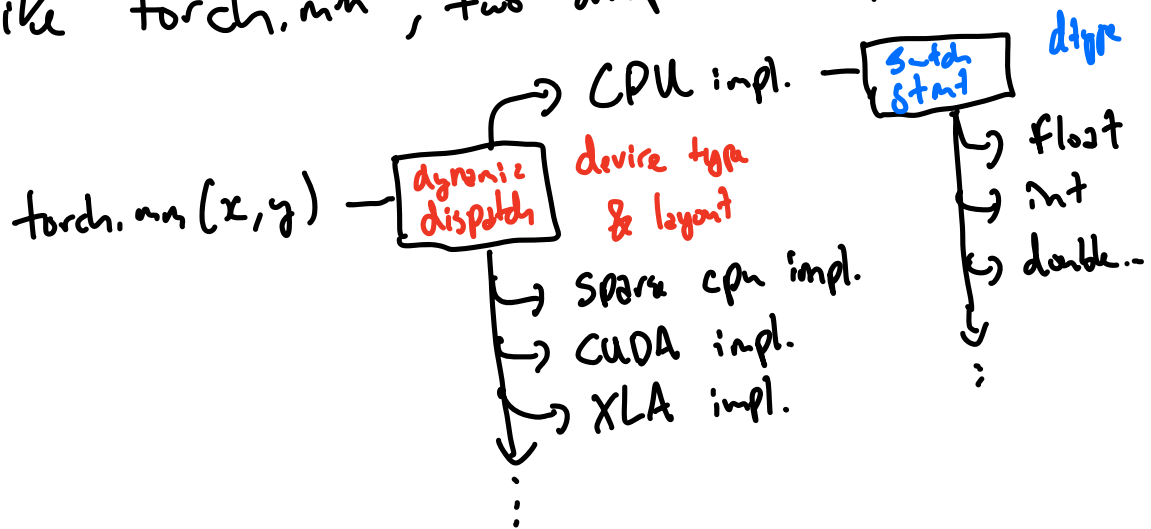
$$\text{offset} = 0$$

To enable these views / ability to generate these sub-tensors, we decouple the data from the tensor obj, having the tensor manage the "view", that holds a ref to the

"storage" object which holds the data, dtype, len, etc.
So in Pytorch, every Tensor contains a storage object, while many Tensors may actually refer to the same storage

Tensor operations:

At an abstract level, when you call something like `torch.mm`, two dispatches happen:



Dynamic Dispatch (Virtual function call)

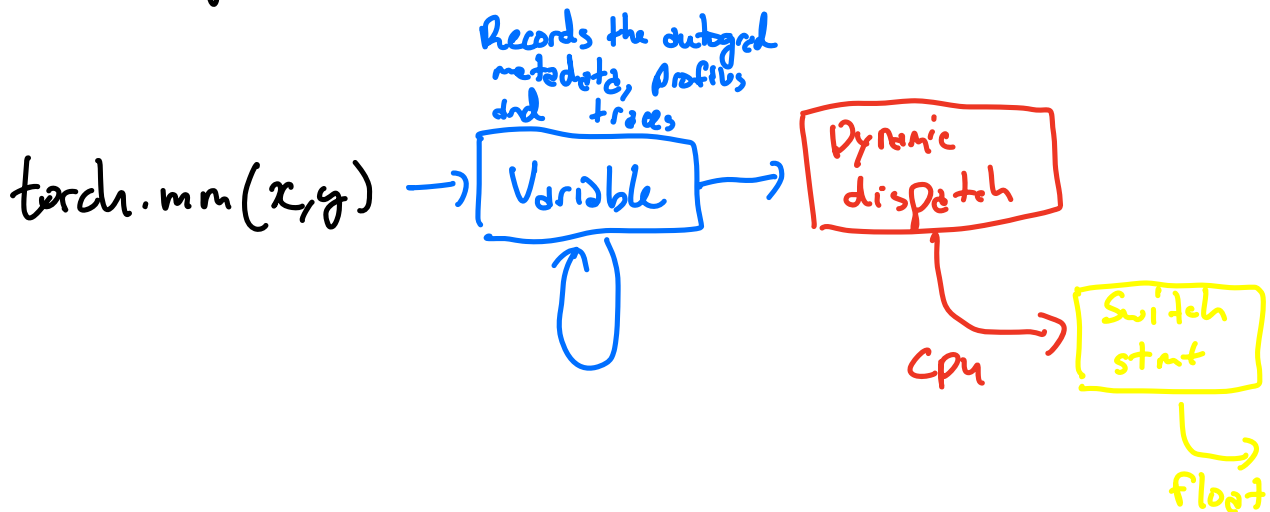
- For the first dispatch, we dynamically (at run-time) select the function that corresponds to the **device type & layout**, calling a virtual function that corresponds

Switch Statement

- Then we select and call the kernel fn corresponding to the particular type

Autograd

- Automatically generates the necessary backward pass calculations without the need to specifically call any of these
- Achieves this by adding metadata to the Tensor (actually wraps Tensor with a Variable class that supports the Metadata)
- This adjusts the dispatch graph:



So at the variable level, we update the variable as computations happen.

Importantly, in the metadata we store

Metadata:

- grad;
- shared_ptr grad_fn;
- weak_ptr grad_accumulator;
- requires_grad;